



資料結構

Data Structure

Lab 09

姓名：林莉婕
學號：112AC1015

Lab01-Ex1. 01

Briefly describe the problem. (題目簡述)

eg. Count operations in Find Maximum code as comments.

Code

```
#include <iostream>
using namespace std;

// 定義樹的節點類別
class TreeNode {
public:
    int data;           // 節點儲存的資料
    TreeNode* left;     // 指向左子節點的指標
    TreeNode* right;    // 指向右子節點的指標

    // 節點建構子：設定資料並初始化左右指標為空
    TreeNode(int val) : data(val), left(nullptr), right(nullptr) {}
};

// 定義二元搜尋樹類別
class BinarySearchTree {
private:
    TreeNode* root;    // 樹的根節點指標

    // --- 私有的遞迴插入方法 ---
    TreeNode* insert(TreeNode* node, int val) {
        if (node == nullptr) {
            return new TreeNode(val);
        }

        if (val < node->data) {
            node->left = insert(node->left, val);
        }
        else if (val > node->data) {
            node->right = insert(node->right, val);
        }
        return node;
    }

    //私有的遞迴刪除方法 ---
    TreeNode* deleteNode(TreeNode* node, int key) {
        if (node == nullptr) return nullptr; // 沒找到目標節點

        // 搜尋目標節點
        if (key < node->data) {
            node->left = deleteNode(node->left, key);
        }
        else if (key > node->data) {
            node->right = deleteNode(node->right, key);
        }
        else {
            //只有一個子節點或沒有子節點
            if (node->left == nullptr) {
                TreeNode* temp = node->right;
                delete node;
            }
```

```

        return temp;
    }
    else if (node->right == nullptr) {
        TreeNode* temp = node->left;
        delete node;
        return temp;
    }

    // 找到右子樹中的最小值節點（中序後繼者）來替換
    TreeNode* temp = findMin(node->right);
    node->data = temp->data; // 複製值
    // 刪除該中序後繼者節點
    node->right = deleteNode(node->right, temp->data);
}
return node;
}

//尋找子樹中的最小值節點
TreeNode* findMin(TreeNode* node) {
    while (node->left != nullptr) node = node->left;
    return node;
}

// --- 私有的遞迴搜尋方法 ---
bool search(TreeNode* node, int target) {
    if (node == nullptr) return false;
    if (node->data == target) return true;
    else if (target < node->data) return search(node->left, target);
    else return search(node->right, target);
}

// --- 私有的中序遍歷方法 ---
void inorderTraversal(TreeNode* node) {
    if (node == nullptr) return;
    inorderTraversal(node->left);
    cout << node->data << " ";
    inorderTraversal(node->right);
}

public:
    BinarySearchTree() : root(nullptr) {}

    void insert(int val) {
        root = insert(root, val);
    }

    // 公開的刪除方法
    void remove(int val) {
        root = deleteNode(root, val);
    }

    bool search(int target) {
        return search(root, target);
    }

    void displayInOrder() {
        cout << "樹的中序遍歷結果（排序後） : ";
    }

```

```

        inOrderTraversal(root);
        cout << endl;
    }
};

int main() {
    BinarySearchTree bst;

    int values[] = { 7, 1, 4, 2, 8, 13, 12, 11, 15, 9, 5 };
    int n = sizeof(values) / sizeof(values[0]);

    for (int i = 0; i < n; ++i) {
        bst.insert(values[i]);
    }

    int key;
    cout << "請輸入一個鍵值 : ";
    cin >> key;

    if (bst.search(key)) {
        cout << "值 " << key << " 已經存在於樹中。" << endl;
    }
    else {
        cout << "值 " << key << " 不存在，將加入樹中。" << endl;
        bst.insert(key);
    }

    bst.displayInOrder();

    // 示範刪除功能 (Q1 測試)
    cout << "\n請輸入一個要刪除的鍵值 : ";
    cin >> key;
    if (bst.search(key)) {
        bst.remove(key);
        cout << "已刪除 " << key << endl;

        /*
            Q3: 刪除圖示說明 (以刪除 13 為例，會有兩子節點的情況)
            刪除前 13 是 12 和 15 的父節點。
            刪除後會找 13 右子樹最小值 (15) 或是左子樹最大值來替換。
        */

        bst.displayInOrder();
    } else {
        cout << "找不到該值，無法刪除。" << endl;
    }

    return 0;
}

```

```

//私有的遞迴刪除方法 ---
TreeNode* deleteNode(TreeNode* node, int key) {
    if (node == nullptr) return nullptr; // 沒找到目標節點

    // 搜尋目標節點
    if (key < node->data) {
        node->left = deleteNode(node->left, key);
    }
    else if (key > node->data) {
        node->right = deleteNode(node->right, key);
    }
    else {
        //只有一個子節點或沒有子節點
        if (node->left == nullptr) {
            TreeNode* temp = node->right;
            delete node;
            return temp;
        }
        else if (node->right == nullptr) {
            TreeNode* temp = node->left;
            delete node;
            return temp;
        }
        // 兩個子節點都有
        // 找到目標節點，且該節點有兩個子節點
        // 找到目標節點，且該節點有兩個子節點
        // 找到目標節點，且該節點有兩個子節點
    }
}

```

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

ample

請輸入一個要搜尋/新增的鍵值：14

值 14 不存在，將加入樹中。

樹的中序遍歷結果（排序後）：1 2 4 5 7 8 9 11 12 13 14 15

請輸入一個要刪除的鍵值：11

已刪除 11

樹的中序遍歷結果（排序後）：1 2 4 5 7 8 9 12 13 14 15

aptflin@linpipiaipupudeMacBook-Air Downloads %